

Simulador XG-PON: IPACT vs GIANT vs QoSDBA

Cumplimiento de SLA en una red de acceso óptica

David Retuerto José Vega Matías Perelli

TEL-341 Simulación de Redes
Universidad Técnica Federico Santa María

2026

Agenda

- 1 Introducción
- 2 El problema
- 3 La red simulada
- 4 T-CONTs
- 5 El SLA
- 6 El problema del DBA
- 7 El simulador
- 8 Experimentos
- 9 Resultados
- 10 Conclusiones

Proyecto 2 (Fase 2): simulamos GPON ITU-T G.984, 32 ONUs, comparando un DBA proporcional contra uno con QoS por T-CONT.

Pivote a esta Fase 3 (reunión con la profesora, 9/6/2026):

- Migrar a **XG-PON1 (ITU-T G.987)**, el sucesor 10G de GPON
- Reducir a **8 ONUs idénticas** y exigir una meta de SLA explícita para VoIP (≤ 2 ms)
- Comparar **IPACT** (polling de EPON, usado a propósito como referencia) contra dos algoritmos nativos de PON (**GIANT**, **QoSDBA**)

Pregunta que motiva todo el proyecto: ¿el algoritmo de DBA importa para que una llamada de voz cumpla su SLA de latencia?

Por qué hace falta un algoritmo

Una PON conecta una central (OLT) con varios usuarios (ONUs) usando un solo cable de fibra, repartido por un splitter pasivo.

- Bajada (OLT a ONUs): no hay problema, la señal le llega a todos.
- Subida (ONUs a OLT): todas las ONUs comparten el mismo canal, solo una puede transmitir a la vez.

Si por ese mismo canal compiten cosas muy distintas, como una llamada de voz y una descarga pesada, alguien tiene que decidir quién transmite y en qué momento. Esa decisión es el corazón de este proyecto.

La red que simulamos: XG-PON1

Estándar ITU-T G.987, el sucesor de GPON.



- Upstream: 2.48832 Gbps
- Trama: 125 μ s (8000 por segundo)
- 38880 bytes disponibles por trama
- Delay de propagación: 100 μ s

T-CONTs: qué son y cuáles usamos

En vez de tener una sola cola para todo el tráfico de una ONU, GPON y XG-PON definen **T-CONTs** (Transmission Containers): una cola distinta por tipo de servicio, cada una con su propia prioridad.

T-CONT	Tráfico	Tasa	Paquete
T-CONT1 (VoIP)	CBR (fijo)	1 Mbps	160 B
T-CONT2 (Video)	Poisson	40 Mbps	1000 B
T-CONT4 (Datos)	Pareto ($\alpha=1.5$)	200 a 800 Mbps ¹	1400 B

¹Varía según el escenario de carga, ver diseño experimental

El SLA: qué es y por qué importa

Un SLA acá es simple: una cota máxima de latencia. Si un paquete llega más tarde que esa cota, se considera que el servicio falló para ese paquete.

T-CONT	Cota	Origen
T-CONT1 (VoIP)	≤ 2 ms	Pedido explícito de la profesora
T-CONT2 (Video)	≤ 20 ms	Meta razonable del equipo
T-CONT4 (Datos)	≤ 500 ms	Cota laxa, solo de referencia

La que de verdad nos interesa demostrar es la de T-CONT1. Las otras dos no son normas del estándar, son metas que pusimos nosotros para tener algo con qué comparar.

Dynamic Bandwidth Allocation

Si cada ONU transmitiera cuando quisiera, habría choques en el canal. Alguien tiene que decidir, trama por trama, quién transmite y cuánto.

Esa decisión se llama **DBA** (Dynamic Bandwidth Allocation), y es la pregunta de investigación real de este proyecto: ¿qué tan bien distintos algoritmos de DBA logran cumplir el SLA de voz?

Broadcast (SR-DBA)

usado por GIANT y QoSDBA

- La OLT calcula la asignación de las 8 ONUs de una vez
- La manda en un solo mensaje (BWmap), cada 125 μ s exactos

Polling

usado por IPACT

- La OLT le habla a una ONU a la vez, en ronda
- El ciclo completo dura lo que se necesite: entre 8 y 1008 μ s

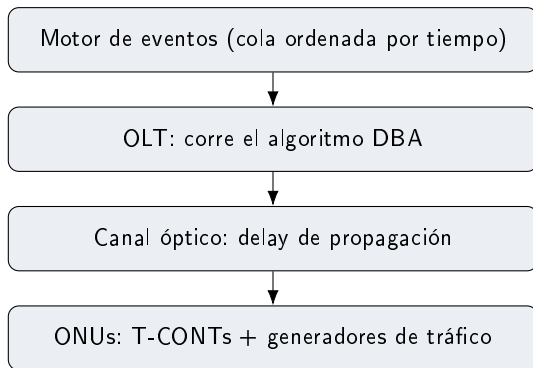
Los 3 algoritmos comparados

Algoritmo	Mecanismo	Trama / ciclo
IPACT	Sondea ONU por ONU, otorga el mínimo entre la demanda y un máximo fijo	Variable: 8 a 1008 μs
GIANT	T-CONT1 reservado siempre, T-CONT2 y T-CONT4 con contadores de turno	Fija: 125 μs
QoSDBA	Prioridad estricta T-CONT1 > T-CONT2 > T-CONT4	Fija: 125 μs

IPACT es de EPON, otro estándar. Lo usamos a propósito como punto de comparación, no porque XG-PON lo use de verdad.

Arquitectura del simulador

Python 3 puro, sin frameworks de simulación (sin SimPy, sin OMNeT++): solo `heapq` de la librería estándar para la cola de eventos.



OLT y ONUs se hablan en los dos sentidos a través del canal: asignación hacia abajo, datos y reportes hacia arriba. El motor de eventos es lo que hace que todo eso pase en el momento correcto.

Entidades y atributos:

- **OLT**: reportes recibidos por ONU, número de trama actual
- **ONU**: un TCont por tipo (1, 2, 4)
- **TCont**: buffer FIFO, bytes encolados, contadores de paquetes/bytes generados y descartados
- **Paquete**: ONU de origen, tipo de T-CONT, tamaño, instante de creación
- *Solo en GIANT*: contadores de turno Sl_{max}/Sl_{min} por ONU. *Solo en IPACT*: índice de la ONU que se está sondeando

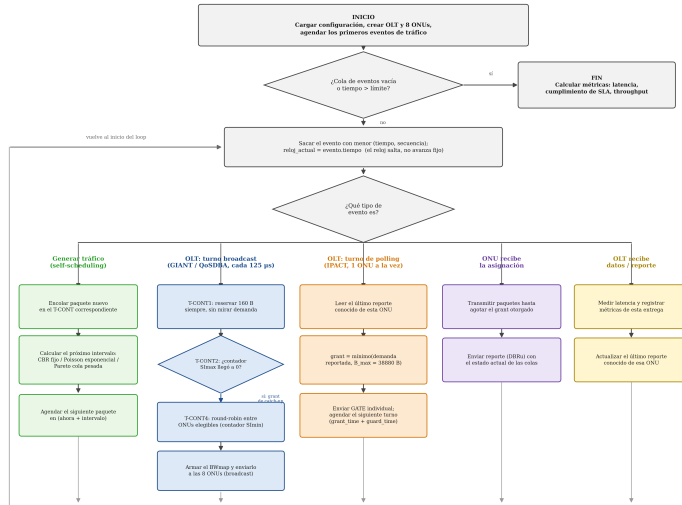
Estado del sistema en el instante t : ocupación de cada buffer T-CONT de cada ONU, último reporte (DBRu) conocido por la OLT de cada ONU, y los contadores de turno/posición de polling vigentes. Ese estado es lo único que el DBA necesita para decidir el siguiente BWmap o GATE.

Eventos futuros (FEL) y condición de término

Evento	Dispara
ONU_GENERATE_TRAFFIC	encola paquete, se reagenda a sí mismo
OLT_GENERATE_BWMAP	DBA broadcast (GIANT/QoSDBA), cada $125 \mu s$
ONU_RECEIVE_BWMAP	ONU transmite + envía DBRu
OLT_SEND_GATE / POLL_NEXT	DBA polling (IPACT), por ONU
ONU_RECEIVE_GATE	ONU transmite + envía DBRu (IPACT)
OLT_RECEIVE_DATA	mide latencia, registra entrega
OLT_RECEIVE_REPORT	actualiza el reporte de esa ONU

Condición de término: tiempo fijo de simulación $[0, T]$ con $T = 10$ s, descartando el primer segundo (*warmup*, 8000 tramas) para evitar medir el transitorio de buffers vacíos. Cada escenario se corre **10 veces** con semillas distintas (6767-6776) para poder calcular intervalos de confianza.

El loop del simulador, paso a paso



El motor de eventos discretos

No hay un reloj que avanza de a poco. Hay una lista de eventos futuros, ordenada por tiempo, y el simulador salta directo al siguiente.

```
mientras la cola de eventos no este vacia:
    evento = sacar el evento mas proximo en el tiempo
    si evento.tiempo > tiempo_limite:
        terminar
    reloj_actual = evento.tiempo      # el reloj salta, no avanza fijo
    handler = handlers[evento.tipo]
    handler(evento)                  # puede agendar nuevos eventos
```

Generación de tráfico: self-scheduling

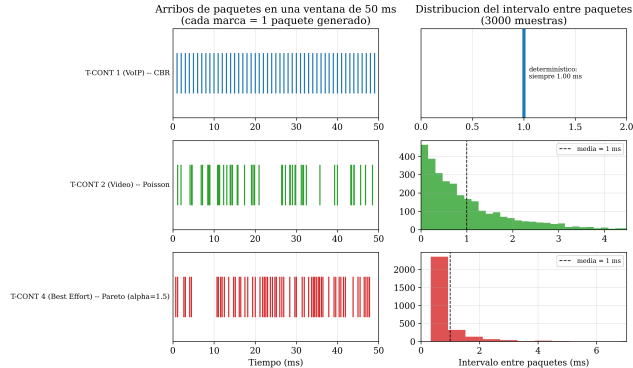
Cada paquete que se genera agenda su propio sucesor. No existe un proceso aparte que esté generando tráfico todo el tiempo.

```
funcion generar_paquete(onu, tcont):  
    encolar nuevo paquete en el buffer de tcont  
    intervalo = tcont.distribucion.siguiente_intervalo()  
    agendar(ahora + intervalo, generar_paquete, onu, tcont)
```

- CBR (T-CONT1): intervalo siempre igual
- Poisson (T-CONT2): intervalo aleatorio, sin memoria del anterior
- Pareto (T-CONT4): intervalo de cola pesada, genera rachas de paquetes

Cómo se ve el tráfico simulado

Como simula el tráfico cada T-CONT (tasas normalizadas a la misma media = 1 ms, solo para esta figura)



Izquierda: línea de tiempo de arribos (cada marca es un paquete). Derecha: histograma del intervalo entre paquetes. Las 3 tasas se normalizaron a la misma media solo para esta figura, para comparar la forma de cada distribución y no la velocidad.

Entradas del simulador: de dónde vienen

- **Parámetros físicos** (tasa, trama, bytes/trama, delay) → **ITU-T G.987.1/2/3**, sin inventar valores
- **Tasas de tráfico T-CONT1/T-CONT2** → heredadas de Fase 2, T-CONT2 escalado $\times 8$ para mantener el mismo % de capacidad (12.86 %) que con 32 ONUs
- **Cargas de T-CONT4** (200/400/800 Mbps por ONU) → barrido propio del equipo, mismas razones de sobrecarga (64 %/129 %/257 %) que en Fase 2
- **Tabla de SLA** → $T\text{-CONT1} \leq 2$ ms es pedido explícito de la profesora; T-CONT2/T-CONT4 son metas razonables propuestas por el equipo, no normas del estándar
- **Contadores de GIANT** ($S_{\text{max}}=8$, $S_{\text{min}}=32$ tramas) → interpretación propia de la semántica de service-interval de G.987.3, el estándar no fija un valor único

- 3 algoritmos (IPACT, GIANT, QoSDBA) $\times$ 3 cargas de T-CONT4 (200, 400 y 800 Mbps por ONU, entre 64 % y 257 % de la capacidad disponible)
- 10 repeticiones por escenario, 10 segundos simulados cada una (con 1 segundo de calentamiento que no se mide)
- Se mide latencia, throughput, pérdida y cumplimiento de SLA por cada ONU y cada T-CONT

Validación: cotas teóricas vs. simulación

No es una cola M/M/1 clásica (los grants son deterministas o por contador), pero sí hay 2 cotas cerradas que se pueden derivar y contrastar directamente:

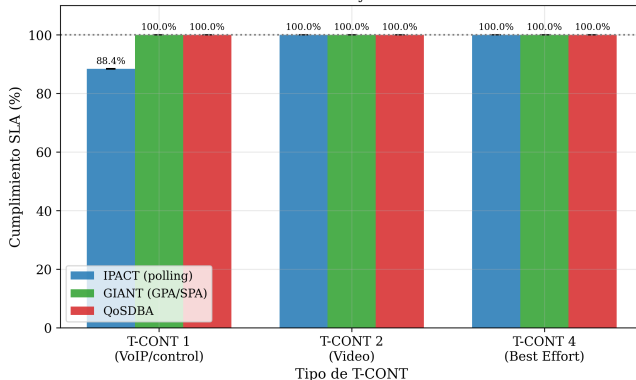
(a) Ciclo de polling IPACT: $[N \cdot t_{guard}, N \cdot (B_{max} \cdot 8/R + t_{guard})] = [8 \mu s, 1008 \mu s]$. Medido: máximo exactamente **1008.000 μs** bajo saturación (400/800 Mbps/ONU) — calza exacto con la cota teórica.

(b) Latencia de T-CONT1 en GIANT/QoSDBA (grant fijo incondicional + CBR):

$L_{max} = T_{trama} + T_{prop} + T_{tx} = 125 + 100 + 1,03 = \mathbf{226.03 \mu s}$. Medido: **226.0288 μs** , con **IC95 %=0** (sin aleatoriedad en ese camino) — confirma que la cota es exacta, no una aproximación.

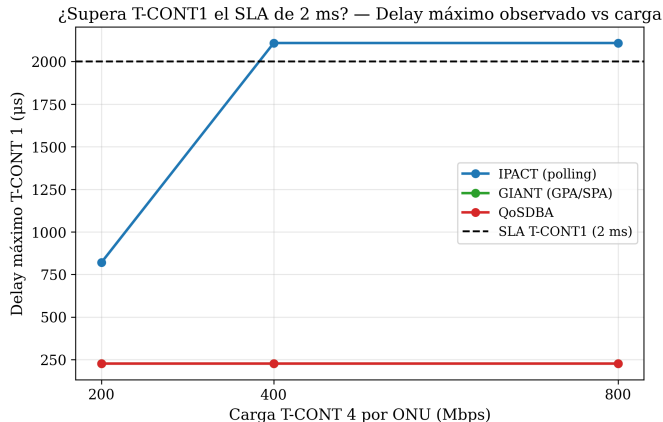
Resultado central: cumplimiento del SLA

Cumplimiento de SLA por T-CONT — Carga BE = 800 Mbps/ONU (sobrecarga ~257%)
T-CONT1 SLA: delay máximo ≤ 2 ms



A 800 Mbps por ONU, IPACT cae a **88.4 %** en T-CONT1 (voz). GIANT y QoSDBA se quedan en 100 % en los 3 T-CONTs.

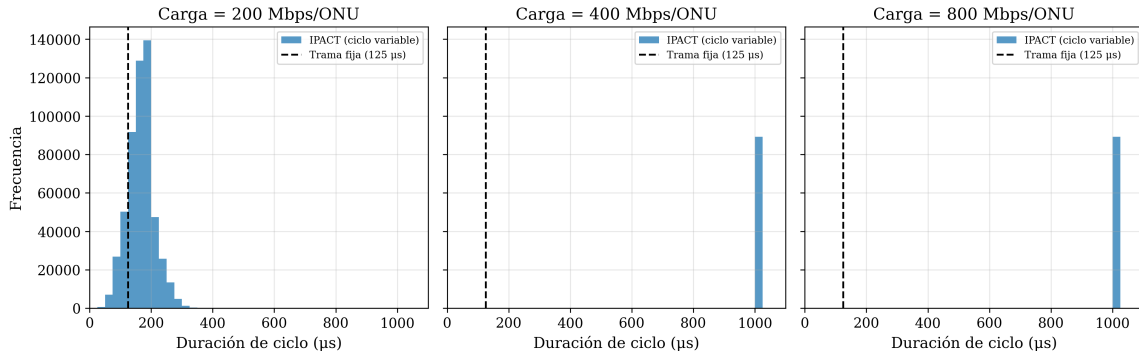
Por qué falla IPACT



A partir de 400 Mbps por ONU, el delay máximo de IPACT cruza la línea de SLA (2 ms). GIANT y QoSDBA se quedan planos en **226 µs**, sin importar la carga.

El mecanismo detrás

Polling de ciclo variable (IPACT) vs trama fija 125 μ s (GIANT/QoSDBA)



Bajo carga alta, el ciclo de IPACT se vuelve constante en **1008 μ s** (el máximo teórico para 8 ONUs). GIANT y QoSDBA usan siempre la trama fija de 125 μ s.

- **10 réplicas i.i.d.** por escenario (semillas 6767-6776), 1 s de *warmup* descartado de 10 s simulados, siguiendo Law & Kelton (2000)
- IC95 % calculado como $\bar{y} \pm 1,96 \cdot s/\sqrt{10}$ sobre las 10 réplicas, reportado para cada métrica en `results.csv`
- **Ejemplo de error relativo** (IC95 / media) en T-CONT1, IPACT a 400 Mbps/ONU: latencia media $1613,08 \pm 0,33 \mu s \rightarrow$ error relativo $\approx 0,02\%$ — variabilidad entre réplicas muy baja
- Donde el algoritmo es determinista (T-CONT1 bajo GIANT/QoSDBA), el IC95 % es exactamente **0**: no hay fuente de aleatoriedad en ese camino, las 10 réplicas son idénticas

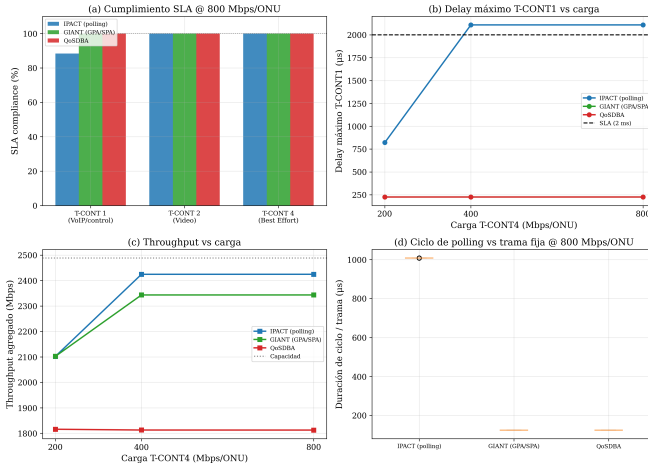
- Cumplir el SLA de voz no es automático, depende de cómo decide el algoritmo de DBA
- Reservar ancho de banda sin condiciones (GIANT, QoSDBA) garantiza el SLA de voz en cualquier escenario que probamos
- Asignar según un reporte de demanda puede fallar justo cuando más se necesita: bajo carga alta y con un reporte desactualizado
- Esto tiene un costo: QoSDBA es más simple pero pierde eficiencia en el tráfico de datos (ver apéndice)

Trabajo futuro: ranging dinámico (ONUs a distancias distintas, RTT variable en vez de fijo), GIANT operando a nivel T-CONT en vez de 1:1 con la ONU, más de un T-CONT del mismo tipo por ONU, y ONUs heterogéneas (distintos perfiles de tráfico simultáneos).

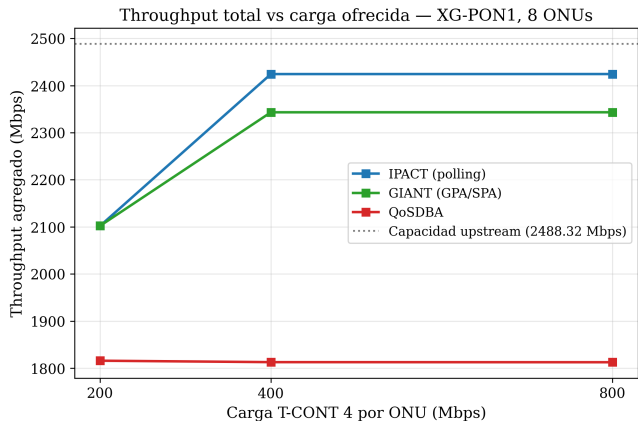
Gracias
¿Preguntas?

Apéndice: dashboard resumen

Fase 3 — XG-PON1, IPACT vs GIANT vs QoSDBA (8 ONUs)



Apéndice: eficiencia agregada



A 800 Mbps por ONU, IPACT y GIANT entregan entre 94 % y 97 % de la capacidad disponible. QoSDBA se queda en 73 %: su reparto proporcional del tráfico de datos deja capacidad sin usar.